Assignment 9: 2D Far-Field Diffraction from a Crossed-Slit Grid

Framework, physics notes and demo plan. Submission due 28 Oct 2026 (email plus a shared Google Drive folder link).

1. What the assignment is really asking

Object: an opaque screen with a grid of open lines (vertical and horizontal slits), slit width a = 5 μm, slit separation 8 μm, lit by a collimated 650 nm laser, with a screen in the far field.

Hard requirements from the handwritten sheet:

- A simulation (Python or MATLAB) that shows the emergence of the 2D far-field pattern.
- Live controls for: slit size, slit separation along the two axes, laser beam size, and the in-plane (xy) angle of the object.
- Every control must update the diffraction pattern live, so the cause-and-effect is visible.

Four interpretation choices you should state openly on a slide (examiners reward this):

1. Separation d means centre-to-centre pitch, so the opaque bars between slits are d − a = 3 μm wide. Add a toggle for edge-to-edge gap in case the professor meant that (pitch would then be 13 μm).
2. The lines are open, so the transmission is a union of two slit arrays, not a product. t = Tx + Ty − Tx·Ty. A product of two 1D gratings would be an array of square holes, which is a different object. This distinction is the heart of the physics.
3. Rotation means rotation about the optical (z) axis, inside the xy plane. Out-of-plane tilt is an optional extra.
4. 'Emergence' means showing how the pattern builds up with propagation distance, from the near field (grid image, Talbot self-images) through the Fresnel region to the Fraunhofer far field. Not only the final picture.

2. What separates a very good submission from an average one

- Correct object model (union of slit arrays) with the physics explained from first principles.

- Live, smooth interaction with a numeric readout of every derived quantity (spot angle, spacing on screen, number of orders, Fresnel number).
- An analytic answer key overlaid on the simulation, so the audience sees the numerical result and the closed-form prediction agree.
- Two independent implementations (Python and MATLAB) cross-checked against each other and against the analytic result.
- A scripted demo that walks through each parameter and states the expected scaling before moving the slider.
- Honest limits: sampling, scalar-theory assumptions, far-field distance.

Priority advice: get one engine flawless first (Python), then build the MATLAB mirror as the independent check. A rock-solid single implementation beats two half-working ones.

3. Software framework (Python and MATLAB together)

3.1 Division of roles

Layer	Python	MATLAB
Live GUI and sliders	Primary (PySide6 + pyqtgraph, or Matplotlib widgets as fallback)	Optional App Designer mirror
Numerical engine	numpy/scipy FFT (fast, multi-threaded)	Independent re-implementation, same algorithm
Analytic answer key	Yes	Yes (symbolic check of the closed form)
Publication-quality static figures	Quick previews	Final report figures
Role in the demo	Real-time driver	Reference solver, cross-validation, second window

3.2 Architecture

```
   GUI (Python)  --params-->  Engine A: Python (numpy FFT)  --intensity-->
display
        |
^
        +--params-->  Engine B: MATLAB (matlab.engine, async)  --intensity---
-+
        |
|
        +--params-->  Analytic model (closed form lattice + envelope) --
overlay

|
                         Comparison panel: |A - B|, A vs analytic, max error
--+
```

One parameter dictionary (a, d for each axis, beam radius, angle, wavelength, screen distance, grid size) is the single contract between all three engines. Same inputs in, same arrays out.

3.3 Three ways to connect MATLAB and Python (pick one as default, keep one as fallback)

1. MATLAB Engine API for Python (recommended). Install the package matching your MATLAB release (pip install matlabengine, choose the version that matches your R2026 release, and check MathWorks' table of supported Python versions first). Start with matlab.engine.start_matlab(), or run matlab.engine.shareEngine in MATLAB and attach from Python with matlab.engine.connect_matlab() so you can also watch MATLAB's own live figure. Call your function with background=True so the GUI never freezes while MATLAB computes.
2. Python from MATLAB (pyenv and py.module.function). Good if you want a MATLAB App Designer front end that calls Python's engine. The reverse direction of option 1.
3. Decoupled processes (fallback, most robust). Python and MATLAB run separately and exchange parameters and result arrays through a local socket (ZeroMQ or plain TCP) or small .mat/.npy files. Slower but cannot be broken by version mismatches.

Benchmark the array round-trip before relying on it. For a 1024 × 1024 double array the transfer is about 8 MB, which is usually fine for 10 to 20 frames per second. If it is not, send only a cropped central window of the far field or use the file/socket route.

3.4 Windows and VS Code setup

- Create a virtual environment inside the project folder and select it as the VS Code interpreter.
- Install numpy, scipy, matplotlib, pyqtgraph, PySide6 (and optionally numba or pyfftw). Pin versions in requirements.txt.
- Install matlabengine into the same environment. Verify with a two-line test: start the engine and call eng.sqrt(4.0).
- Use VS Code's MATLAB extension for editing .m files next to your Python.
- Keep MATLAB figures readable regardless of your editor theme: at the top of every MATLAB script set the figure and axes colours to white with dark text (for example set(groot,'defaultFigureColor','w'); set(groot,'defaultAxesColor','w'); set(groot,'defaultAxesXColor','k'); set(groot,'defaultAxesYColor','k'); set(groot,'defaultTextColor','k')). Image panels showing the laser spot can use a dark or red colormap inside their own axes.

3.5 Suggested project layout

```
diffraction_demo/
  README.md               how to run in 3 commands
  requirements.txt
  params.py               defaults: lambda=650e-9, a=5e-6, d=8e-6, ...
  core_py/
    object_model.py       transmission t(x,y), beam B(x,y), rotation
    propagate.py          Fraunhofer FFT, angular spectrum, Fresnel single-
FFT
    analytic.py           closed-form orders, envelope, missing orders
  core_matlab/
    object_model.m  propagate.m  analytic.m  run_reference.m
  bridge/
    matlab_link.py        engine wrapper, async calls, fallback socket
  gui/
    app.py                sliders, panels, presets, record button
  tests/
    test_limits.py        single slit, a=d, 1D grating, symmetry, Parseval
  report/                 figures and PDF
```

3.6 Keeping it real time

- Two quality modes: interactive while dragging (N = 1024), full quality on slider release (N = 2048 or 4096). Debounce slider events and drop stale frames.
- Use complex64 and scipy.fft with workers=-1. 2D FFTs of 1024^2 or 2048^2 take tens of milliseconds.
- Zero-pad only in the release mode (to smooth spot shapes), not while dragging.
- Optional speed-ups: cupy or MATLAB gpuArray for the FFT, and numba for building the object mask.

4. The numerical engine step by step

1. Grid. Choose N × N samples at pitch dx. Field of view L = N·dx.
2. Object. Rotate coordinates, $x' = x\cos\theta + y\sin\theta$ and $y' = -x\sin\theta + y\cos\theta$. A vertical-slit array is open where |x′ reduced to the nearest period centre| < ax/2, a horizontal array where |y′ reduced| < ay/2. Combine with $t = 1 - (1 - Tx)(1 - Ty)$. Use soft (area-coverage) edges of about one pixel width so rotated edges do not alias.
3. Beam. Fixed in the lab frame, $B(x,y) = \exp(-(x^2 + y^2)/w^2)$ for a Gaussian of $1/e^2$ intensity radius w (the object rotates under it). Offer a circular top-hat as an alternative.
4. Far field (Fraunhofer). U = fftshift(fft2(ifftshift(t·B))). Intensity $I = |U|^2$. Frequency axis fx = k/(N·dx), direction sine $\sin\theta x = \lambda \cdot fx$. Plot the main view against $\sin\theta x$, $\sin\theta y$ (no distortion), with a toggle to remap to the physical screen $X = z\cdot\tan\theta x$ for a chosen z.
5. Emergence in z. Use the angular spectrum method for short distances: multiply FFT(U0) by $\exp(i\cdot 2\pi\cdot z\cdot\mathrm{sqrt}(1/\lambda^2 - fx^2 - fy^2))$ and cut evanescent components. For long distances switch to the single-FFT Fresnel transform, FFT of $U0\cdot\exp(i\pi(x^2 + y^2)/(\lambda z))$, whose output pixel pitch is λz/(N·dx) and which tends exactly to the Fraunhofer result as z grows. (The angular spectrum method on a finite window wraps around once spreading orders leave the window; that is why the hybrid is needed.)
6. Analytic overlay. Compute the order positions, the amplitudes from the table in section 5.3, and the envelope nulls from the formulas, and draw them on top of the simulation.

Sampling guide for λ = 650 nm, a = 5 µm, d = 8 µm:

Mode	N	dx	Field of view	Highest order resolved (Nyquist)	Largest sensible beam radius
Interactive	1024	0.5 µm	512 µm	±8	about 100 µm

Mode	N	dx	Field of view	Highest order resolved (Nyquist)	Largest sensible beam radius
Quality	2048	0.4 μm	819 μm	±10	about 200 μm
Export	4096	0.3 μm	1229 μm	all 12 propagating orders	about 300 μm

Rule of thumb: keep L at least 4 beam radii (so the Gaussian is not clipped), and at least 10 pixels across the 5 μm slit.

5. The physics you must be able to explain

Present these in this order. Each subsection ends with the line you can say to the professor.

5.1 Why the far field is a Fourier transform

Huygens: every open point of the screen is a secondary source. In the far field the path difference from a point (x,y) is linear in x and y, so the sum over all sources is a Fourier integral:

$U(\theta x, \theta y) \propto \int\int t(x,y)\cdot B(x,y)\cdot \exp[-i\cdot k\cdot(x\cdot \sin\theta x + y\cdot \sin\theta y)]\, dx\, dy$, with spatial frequency $fx = \sin\theta x/\lambda$.

Equivalent picture: the object is a superposition of spatial frequencies, and each frequency is a plane wave leaving in one specific direction. That picture gives the grating equation for free.

Say: 'The far-field screen is a map of the object's spatial-frequency content, direction by direction.'

5.2 The object function

Let Tx(x) be the array of vertical slits (width ax, pitch dx) and Ty(y) the array of horizontal slits (width ay, pitch dy). The grid of open lines is

$t(x,y) = Tx + Ty - Tx\cdot Ty$,

which is the union of the two arrays (the overlap squares are counted once). Open-area fraction $= 1 - (1 - ax/dx)(1 - ay/dy)$. With the given numbers it is $1 - (3/8)^2 = 0.859$, so 86 percent of the light passes.

5.3 Closed-form far field of an infinitely wide beam

Write fx = ax/dx and fy = ay/dy (fill factors) and sinc(υ) = sin(πυ)/(πυ). Using the Fourier series of each slit array, the field is a lattice of spots at the directions given by the grating equations

sinθx = m·λ/dx, sinθy = n·λ/dy (integers m, n),

with amplitudes:

Order (m, n)	Amplitude
(0, 0)	fx + fy − fx·fy
(m ≠ 0, 0)	fx·sinc(m·fx)·(1 − fy)
(0, n ≠ 0)	fy·sinc(n·fy)·(1 − fx)
(m ≠ 0, n ≠ 0)	− fx·fy·sinc(m·fx)·sinc(n·fy)

(Phases depend on where you put the origin; intensities do not.)

How to read it: the first two terms of t each give a 1D diffraction pattern, one along each axis (the 'cross'); the product term supplies the off-axis lattice spots. The envelope is a product of two single-slit sinc² functions, one per axis.

Babinet's principle gives the same answer from the opposite side and is a strong cross-check to show: the grid equals a uniform open screen minus an array of opaque squares of side b = d − a = 3 μm. So the pattern is the undiffracted beam minus the lattice pattern of the opaque squares, with a sinc envelope set by b. The intensities agree with the table above.

Say: 'Spot positions are set by the pitch, spot heights by the slit width, and the 2D lattice is just the product structure of the two axes.'

5.4 The numbers for this assignment (your answer key)

λ = 650 nm, a = 5 μm, d = 8 μm on both axes.

- Fill factor a/d = 0.625. Open-area fraction 0.859.
- First-order angle: sinθ = λ/d = 0.08125, so θ1 ≈ 4.66°.
- Spot spacing on a screen at z = 1 m: λz/d ≈ 8.1 cm.
- Propagating orders: |m| ⩽ d/λ = 12.3, so 12 per side per axis. Beyond that the order is evanescent.

- Single-slit envelope first null: $\sin\theta = \lambda/a = 0.13$, so $\theta \approx 7.47°$, which is at order $m = d/a = 1.6$.
- Missing orders (where an envelope null coincides with an order): $m \cdot a/d$ integer, so $m = \pm 8$ (and ± 16 if they existed). Orders 1 to 7 are visible, order 8 is dark.
- Approximate intensity relative to the central spot: $(\pm 1,0) \approx 0.016$, $(\pm 1,\pm 1) \approx 0.010$, $(\pm 2,0) \approx 0.0024$, $(\pm 3,0) \approx 3 \times 10^{-4}$, $(\pm 4,0) \approx 0.0012$.

Important consequence: the zero order carries almost all of the light, and the higher orders are 1 to 3 orders of magnitude weaker. Your display needs a log or gamma control, otherwise the audience sees one bright dot. Explain this on the slide, it is physics (86 percent open area), not a bug.

These values are your built-in answer key: simulated spot positions should match to within a pixel and spot heights to within about 1 percent for a wide beam.

5.5 What each control does (predicted scaling, state it before you move the slider)

Control	Effect on the pattern	Formula
Slit width a	Only the envelope changes. Spot positions fixed. Narrower slit gives a wider envelope and brighter high orders. Missing orders move.	nulls at $\sin\theta = k\lambda/a$
Pitch d	Spot spacing scales as 1/d. Envelope fixed in angle. Larger d packs more orders under the envelope.	$\sin\theta = m\lambda/d$
ax ≠ ay, dx ≠ dy	Rectangular, anisotropic lattice. A larger pitch on one axis gives tighter spacing on that axis of the screen.	independent per axis
Beam radius w	Every spot is a blob of angular half-width $\lambda/(\pi\cdot w)$ ($1/e^2$ intensity). Positions and envelope unchanged. Small w makes spots merge.	$\theta_b = \lambda/(\pi\cdot w)$
Rotation angle θ	The whole pattern rotates rigidly by the same angle (Fourier rotation theorem). Beam spot shape stays round if the beam is round.	$\mathcal{F}\{t(R\cdot r)\} = \mathcal{F}\{t\}(R\cdot f)$
Wavelength	Spacing and envelope both scale with λ.	$\theta \propto \lambda$
Screen distance z	In the far field only the scale of the screen coordinates changes.	$X = \lambda\cdot z\cdot f_x$

Beam detail: orders overlap when the spot width $\lambda/(\pi\cdot w)$ approaches the order spacing λ/d, that is when w is about $d/\pi \approx 2.5$ μm. For w = 100 μm the spots are about 2 mm wide on a screen 1 m away versus 8.1 cm spacing, which is crisp. A top-hat circular beam of radius R gives Airy blobs instead, first null at $\sin\theta = 0.61\cdot\lambda/R$.

5.6 Near field, Talbot effect, and when 'far field' begins

This is the physics behind 'emergence'. Use w = 100 μm as a worked example.

- z below $d^2/\lambda \approx 0.1$ mm: near field. You see a geometric shadow of the grid with edge ringing.
- $z = 2d^2/\lambda \approx 197$ μm (Talbot length): self-images of the grid reappear, a fractional Talbot plane shows a half-period-shifted image. Only visible because the grid is periodic and the beam is wide.

- z of order $2 \cdot w \cdot d/\lambda \approx 2.5$ mm: the diffraction orders have walked apart by more than the beam width and begin to separate into individual spots.
- True far field, z much larger than $2D^2/\lambda$ with $D = 2w$, about 12 cm here (for w = 1 mm it would be about 12 m). Equivalent statement: Fresnel number $w^2/(\lambda z)$ much smaller than 1. Show the Fresnel number live and colour it green once it drops below 0.1.

Note for the demo: if you want the far field on a bench in a short distance, a lens moves it to the focal plane. Mention it.

Say: 'Far field is not a place, it is a condition: the Fresnel number must be small.'

5.7 Rotation and the reciprocal lattice

The real-space lattice vectors are (dx, 0) and (0, dy), rotated by θ. The diffraction pattern is the reciprocal lattice, with vectors λ/dx and λ/dy rotated by the same θ. Short real-space periods map to long reciprocal vectors, which is why a finer grid spreads the pattern out. A small inset drawing real and reciprocal lattices side by side makes this intuitive.

5.8 Scalar-theory validity and limits (say these before you are asked)

- Scalar diffraction is accurate when features are much larger than λ. Here a = 5 μm ≈ 7.7 λ, so it is acceptable, but polarization and edge effects are neglected. A rigorous treatment would need vector (RCWA or FDTD) methods.
- The screen is assumed infinitely thin and perfectly opaque; the beam is perfectly collimated and fully coherent.
- The paraxial and small-angle approximations are weak for high orders. At order 5, $\sin\theta = 0.41$ but $\tan\theta = 0.44$, about 9 percent apart, so plotting against direction sines (not screen position) avoids distortion. The obliquity factor (about $\cos\theta$) slightly dims high orders.
- The grid is assumed much larger than the beam; add an optional finite number of slits to show the N-slit sharpening.

6. What to show on the visualisation

6.1 Panel layout (2 × 3 grid)

A. Object plane. The transmission map (zoomed to a few periods, with a toggle for the whole illuminated area) with the beam outline overlaid and the rotation visible live.

B. Far-field 2D intensity (the main attraction). Axes in sinθ (or degrees), plus the physical screen scale for the chosen z. Controls: linear, log and gamma, a 'laser-photo' red colormap, and brightness gain. Overlays you can toggle: circles at the predicted lattice points, dashed lines at the sinc envelope nulls, crosses on missing orders.

C. Line cuts. Intensity along the fx and fy axes through the central order, log scale, simulation (solid) versus analytic (dashed). This is the quantitative proof.

D. Real and reciprocal lattice sketch with the two periods and their rotation.

E. Emergence strip or slider. Same object at z = 0, 0.1 mm, Talbot length, 1 mm, 10 mm, 100 mm and infinity, plus a play button that sweeps z. Show the live Fresnel number and a similarity score to the Fraunhofer pattern that rises toward 1.

F. Readout panel. All derived numbers update live: first-order angle, spot spacing at the screen, number of propagating orders, visible and missing orders, envelope null angle, spot width, open-area fraction, zero-order share of power, and Fresnel number.

6.2 Interaction features that make the cause and effect obvious

- Sliders with numeric entry for ax, ay, dx, dy (plus a 'link axes' tick for symmetric changes), beam radius, rotation angle, wavelength, screen distance.
- A 'freeze reference' button that holds the current pattern as a ghost overlay (or difference view), so the audience sees exactly what the last slider move changed.
- Play buttons that animate one parameter through a sweep (pitch, slit width, beam radius, angle).
- Reset to the assignment defaults (650 nm, 5 μm, 8 μm).
- Presets: single slit, double slit, N slits, 1D grating, crossed-slit grid (the assignment), square-hole array (the product object, for contrast), rotated grid.
- Engine switch: Python only, MATLAB only, or both with a difference map and the maximum error shown.
- A record/screenshot button for the report and a backup video.

7. Demo script (8 to 10 minutes)

1. State the setup and the four interpretation choices (section 1). Show the object and the default pattern with the answer-key overlay on.
2. Emergence: sweep z from the near field to infinity. Narrate Talbot images, order separation, and the Fresnel number crossing the far-field threshold.

3. Pitch sweep: spots move apart as d shrinks. Prove 1/d scaling by reading the spacing from the readout.
4. Slit-width sweep: envelope breathes while spot positions stay put. Show the missing order at m = 8 for a = 5 μm, then set a = d/2 = 4 μm to show the even orders vanishing along the axes.
5. Beam-size sweep: spots sharpen as w increases (1/w), merge as w shrinks.
6. Rotation: rigid rotation of the whole lattice, with the reciprocal-lattice inset.
7. Anisotropy: different ax, ay, dx, dy to give a rectangular lattice.
8. Object type comparison: union grid versus product (square holes) versus the Babinet complement, so the audience sees why the object model matters.
9. Validation: Python versus MATLAB difference map, then line cuts versus analytic.
10. Limits and honesty: scalar theory, sampling, far-field distance.

Questions to prepare for:

- Why is the central spot so dominant? Because the open-area fraction is 86 percent and the zero order is the undiffracted beam.
- Why is the 8th order missing? Because an order lands on a single-slit null when m·a/d is an integer.
- Why not multiply the two 1D patterns? Because the lines are open; the product is a different object (holes).
- What happens for slits smaller than λ? Scalar theory breaks down; only the zero order propagates and polarization dominates.
- What if the screen is close? Then the Fresnel number is not small; show the emergence strip.
- What if the beam is elliptical or tilted? Spot shapes become elliptical, and tilt shifts the whole pattern by the Fourier shift theorem.

8. Validation checklist (put the results in the report)

- Spot positions equal mλ/dx and nλ/dy to within one pixel.
- Spot heights match the analytic table to about 1 percent for a wide beam.
- 1D limit: set ay = dy (fully open along y) and confirm a pure 1D grating pattern.
- a = d gives a single central spot.
- Symmetry: I(−fx, −fy) = I(fx, fy) for a real object.
- Energy conservation (Parseval): total power unchanged by propagation.

- Convergence: doubling N or halving dx does not change the pattern beyond noise.
- No aliasing: rotated edges are smooth, highest order stays below Nyquist.
- Python and MATLAB agree to near machine precision after identical sampling.

9. Deliverables and schedule

Submission package (shared Google Drive folder with view access for anyone with the link, link sent by email):

- Source code for Python and MATLAB, requirements.txt, a README that runs in three commands.
- A 3 to 5 page PDF report with the physics derivation, the answer-key table, validation figures, and screenshots.
- A 2 to 3 minute screen recording of the live demo as a backup in case the live demo fails.
- Test the folder link from a different (logged-out) account before sending.

Suggested plan (today is 22 Oct, due 28 Oct):

Day	Target
Thu 22 Oct	Python core: object, beam, Fraunhofer FFT, analytic table, unit tests
Fri 23 Oct	GUI with all sliders, log view, readout panel
Sat 24 Oct	MATLAB mirror and engine bridge, cross-validation panel
Sun 25 Oct	Emergence (angular spectrum plus Fresnel single-FFT), overlays, presets
Mon 26 Oct	Report, figures, demo video, rehearse the script
Tue 27 Oct	Fresh-machine test, bug fixes, buffer
Wed 28 Oct	Submit early in the day

10. Stretch goals (only after everything above works)

- Out-of-plane tilt of the object (projected pitch d·cosφ and a pattern shift).

- Non-orthogonal grid with an adjustable shear angle between the two slit families.
- Wavelength sweep or white light to show the rainbow lattice.
- Finite object size with an adjustable number of slits.
- Elliptical or off-centre beam.
- Fourier-plane filtering demo (block the zero order and show the image).
- GPU acceleration (cupy or gpuArray).